Project Report

Smart Parking Management System Using Modern Deep Learning, AI-Based Parking Predictions and Automatic License Plate Recognition

Looking for a parking space is a small daily annoyance that adds up. Drivers burn fuel circling the lot, and most car parks still depend on a guard at the gate or a sensor in every bay. In this project we built a parking system that gets by with one cheap camera per gate. An ESP32-CAM at the entry and another at the exit stream video to a single computer. There, YOLO11 picks out the vehicles, a second detector finds the number plate, and a small OCR model trained only on plates reads it. We do not trust a single frame: a plate has to be read twice before anything happens, so a one-off misread cannot open the barrier. A confirmed plate is checked against a registry of vehicles. Stolen, blocked and unregistered cars are refused, while admitted cars get a space number that stays fixed until they leave. The barrier is opened through a relay, and alerts are sent when something needs attention. The system also predicts occupancy an hour ahead, using either a seasonal baseline or an LSTM, and shows everything on a live operator dashboard that can also run full-screen. In tests on six videos (95 plate detections), the plate OCR got 78.7% of the readable plates exactly right, compared with 9.3% for EasyOCR, and took about 25 ms per plate on a laptop CPU. None of the misreads turned into a false entry, because of the consensus and format rules. Both gates handled real footage at the same time without falling behind. On a held-out week of synthetic data, the LSTM forecasts were off by 1.79 spaces on average, against 2.45 for the baseline. Everything runs offline on one machine using open-source software.

Keywords: smart parking, automatic number plate recognition, YOLO11, ESP32-CAM, LSTM forecasting, edge computing

1. Introduction

Anyone who drives in a growing city knows the routine. You reach the car park, find it full or nearly full, and start circling. That costs fuel and time and adds to the traffic on the surrounding roads. Car parks that are run by hand have a problem of their own, since somebody has to sit at the gate, check each vehicle and keep track of how many spaces are left. Smart parking systems try to take over these jobs: they work out which vehicles are inside, tell drivers whether there is room, and control access automatically [1].

Most systems in the literature go about this in one of two ways. The first puts a sensor (infrared, ultrasonic or magnetic) in every space and links the sensors through a microcontroller and Wi-Fi [2, 4, 5]. This is accurate for each individual space, but the cost of installing and maintaining it grows with every space that is added. The second mounts a camera above the lot and classifies each space as free or taken [3, 6]. That only works if the camera can see every space, which is hard to arrange in covered or oddly shaped car parks, and even then the system has no idea *which* vehicle is parked where.

We took a different route, based on a simple observation. In most real car parks, every vehicle has to pass through an entry gate and an exit gate. A camera at each gate can therefore count the vehicles and also identify them by their plates, and occupancy becomes the set of plates that have come in but not yet gone out. Knowing which car is which brings extra benefits that the other approaches cannot easily offer. These include access control for registered, blocked or stolen vehicles, a searchable record of every visit, and the data needed to forecast how full the lot is going to get.

The main contributions of this project are:

(1) A working end-to-end system built around gate cameras. It includes ESP32-CAM firmware that also drives the barrier relay, a real-time recognition pipeline, access control, fixed space assignment, storage, alerts and an operator dashboard. All of it runs on one computer and does not need the internet.

(2) A plate recognition pipeline in three stages (YOLO11 for vehicles, a separate plate detector, and an OCR model trained specifically on plates). Several checks keep misreads out: a plate must be read twice, it must match the expected format, and near misses are matched against plates the system already knows.

(3) A way to run more than one gate in real time on an ordinary CPU. When inference cannot keep up with the video, frames are skipped instead of piling up, and a first-in-first-out lock makes the two gates share the models fairly.

(4) Occupancy forecasting one hour ahead, with either a seasonal baseline or an LSTM, plus a generator of synthetic occupancy data so that the LSTM can be trained before a lot has any real history.

(5) An evaluation on real traffic footage. We measured plate-reading accuracy against hand-labelled crops, timed each processing stage, ran both gates together from start to finish, and measured forecasting error on a held-out week.

The rest of the report is laid out as follows. Section 2 looks at related work, and Section 3 explains how the system is built. Section 4 presents and discusses the results, and Section 5 concludes.

2. Related Work

Biyik et al. [1] surveyed smart parking systems from a technical angle, looking at the sensors, communication methods and software architectures in use, and concluded that Internet of Things (IoT) technology will be central to them. Patil et al. [2] used infrared sensors with an Arduino Uno and an ESP8266 Wi-Fi module to detect cars coming in and going out, and guided drivers to empty spaces. Nadipalli et al. [4] took a similar approach with infrared sensors, a microcontroller and a Wi-Fi modem, but sent updates to a cloud server every 10 seconds. As a result, the whole system stopped whenever the network or the microcontroller went down. Subramanian and Subashini [5] proposed a ticket-based design with sensors in every lane and SMS notifications sent through a mobile app.

Camera-based methods do away with a sensor in each space. Bassam and Samann [3] marked every space with a character and ran optical character recognition (OCR) on an overhead image to see which characters were visible. They reported over 90% accuracy on eight test images. Alsheikhy et al. [6] combined a single overhead camera, some image preprocessing and the AlexNet convolutional neural network [8] to count parked cars and send drivers to lanes with free spaces, and reported over 97% accuracy in MATLAB simulations. Alharbi et al. [7] applied OCR to number plates as part of a web-based reservation system. What these systems have in common is that they either need a view of every space or identify spaces rather than vehicles.

Deep learning has greatly improved automatic number plate recognition (ANPR). Single-stage detectors such as YOLO [9] and its later versions, YOLO11 [10] among them, can detect vehicles in real time, while transformer-based detectors like RT-DETR [11] give up some speed for accuracy. Laroca et al. [12] built a real-time ANPR system out of a cascade of YOLO detectors, and Silva and Jung [13] tackled plates seen from steep angles. For reading the characters there are two options. General scene-text engines such as EasyOCR [15], which relies on the CRAFT text detector [16], work on any kind of text. Plate-specific models such as fast-plate-ocr [14] are trained only on cropped plates.

On the forecasting side, simple seasonal methods are known to be hard to beat on data with daily and weekly cycles [18], and recurrent networks such as long short-term memory (LSTM) [17] can learn both the pattern and how it interacts with the current level. Our work brings these pieces together in a single system that runs at the gates of an ordinary car park.

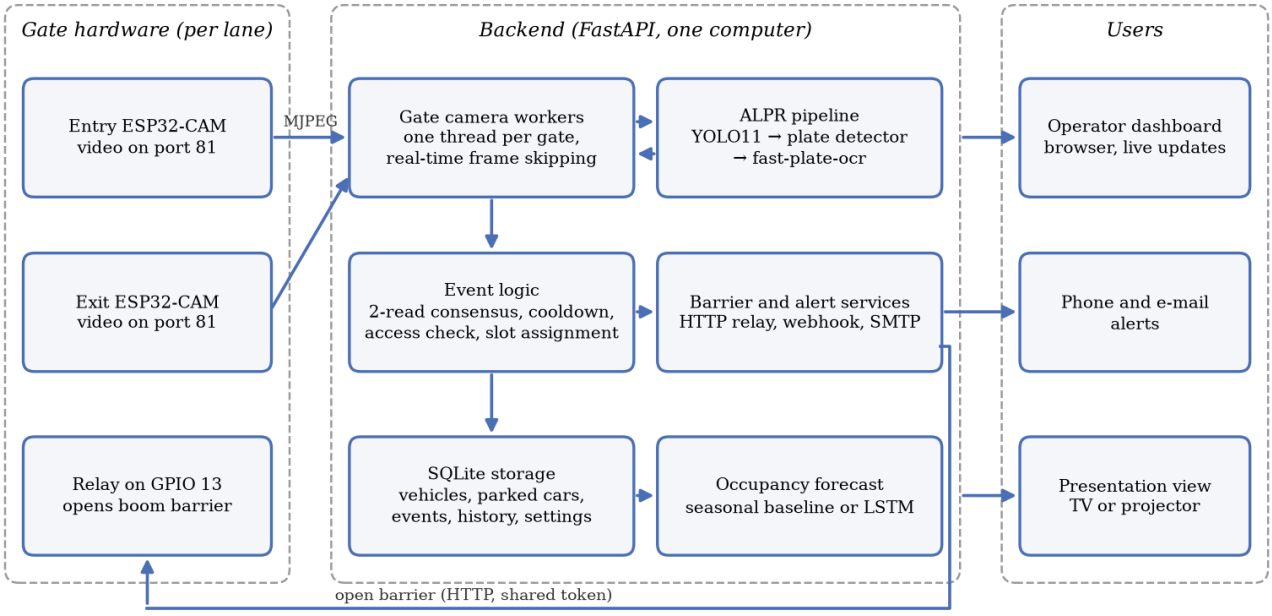


FIGURE 1: System architecture. Each gate has an ESP32-CAM and a barrier relay; one computer runs recognition, event logic, storage, forecasting and the dashboard.

3. The Proposed System

Figure 1 gives an overview of the architecture, which has three parts: the hardware at each gate, a backend running on one computer, and the browser pages that operators use. Figure 2 follows a single analysed frame on its way to becoming a parking event, and Algorithm 1 sets out the same logic as pseudocode.

3.1. Gate Hardware. Each lane has its own AI-Thinker ESP32-CAM board. After joining the local Wi-Fi, the firmware runs two small HTTP servers. Port 81 carries a Motion JPEG stream for the backend, while port 80 handles `/capture` (a single JPEG), `/status` (gate name, uptime and signal strength) and `/open`. We gave the stream its own port for a practical reason. A stream request never finishes, so if it shared a server with `/open`, barrier commands could get stuck behind it. Calls to `/open` have to carry a shared token. The board then switches on a 5 V relay on GPIO 13, whose contacts are wired to the open input of the barrier controller. The main loop switches the relay off again later, so the request returns straight away. If the Wi-Fi connection drops, the backend keeps trying to reconnect to the stream and waits

a little longer after each failed attempt.

3.2. Recognition Pipeline. For each analysed frame, YOLO11n [10], trained on the COCO data set, looks for cars, motorcycles, buses and trucks. A second YOLO model, trained for number plates, finds the plate boxes. Each crop is then read by fast-plate-ocr [14], a compact convolutional model exported to ONNX that gives a confidence for every character, with EasyOCR [15] still available as a fallback. The OCR output is cleaned up next. Everything except capital letters and digits is removed, and any read whose average character confidence is below 0.4 is thrown away. If what is left is very close to a plate the system already knows, either registered or currently parked (a string similarity of 0.82 or more), we take it to be that plate. Otherwise the read has to fit the Indian plate format `^[A-Z]{2}\d{1,2}[A-Z]{0,3}\d{4}$`, as in MH12AB1234. For test videos from other countries there is a looser mode that accepts any 4 to 10 letters and digits.

3.3. From Reads to Events. Two practical problems come up at a gate. Any single frame can be misread, and a car that stops at the barrier stays in view for many frames. We handle these with two rules. The first is that a camera read is ignored until the same plate has been seen twice at that gate within 10 seconds. The second is a cooldown. After an event has been logged for a plate at a gate, further reads of it at the same gate are ignored for 15 s, a value that can be changed from the dashboard. For camera reads the cooldown also covers plates that are one character off (a similarity of 0.85 or more). So if the OCR gets one letter wrong on the car it has just logged, that car is not counted twice.

When a car arrives at the entry, its plate is looked up in the registry. If the vehicle is stolen, blocked or not registered, the barrier stays down and an alert goes out. A registered car is let in as long as there is a free space. It gets the lowest-numbered free space, keeps that number until it leaves, and the barrier opens. If there is no room, the car is turned away. At the exit, the car's space is released and the barrier opens. Occasionally a car leaves without ever having been seen coming in; we log this as a problem so that the operator knows to check the entry camera. Operators can also record an entry or exit by hand whenever a camera misses a car.

3.4. Real-Time Processing. Every gate runs in a thread of its own. The obvious approach, analysing every frame in turn, breaks down as soon as inference takes longer than the gap between frames. A 30-second clip can then take several minutes, and a live stream drifts further and further behind. To avoid this, each worker keeps track of real time. With a video file, it skips ahead using the cheap `grab()` call until it reaches the frame that matches the time that has actually passed. With a live stream, it simply keeps a buffer of one frame. Frames are then analysed at a configurable rate, two per second by default. Because the models are shared, inference goes through a first-in-first-out lock, so the entry and exit gates take turns and neither can hog the models. We also warm the models up as soon as they are loaded; without this, the first car would wait about 8 seconds longer than the rest.

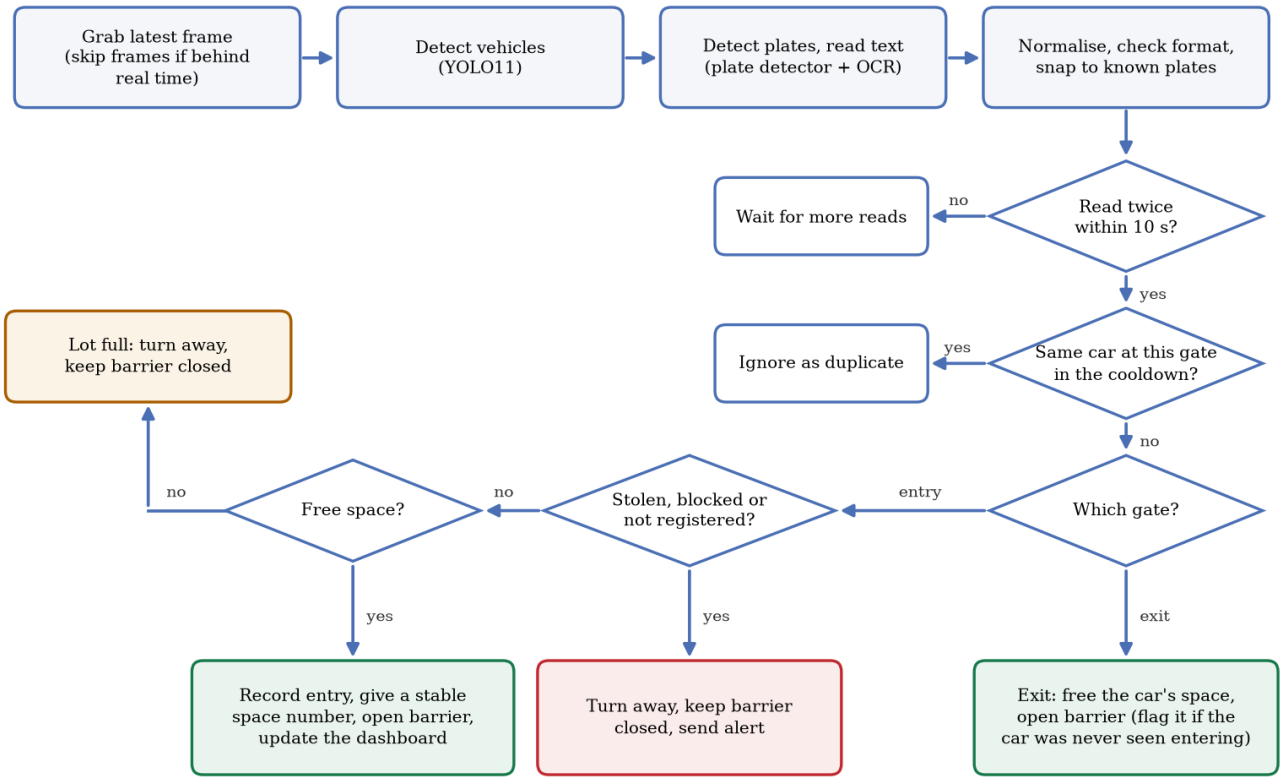


FIGURE 2: Flowchart of the proposed approach, from an analysed frame to a parking event.

3.5. Occupancy Forecasting. We save a snapshot of occupancy every five minutes and after every event. By default, forecasts come from a seasonal baseline. The history is first resampled onto a 15-minute grid. The forecast for a horizon h then mixes two things: the current level L, an exponentially weighted mean of the last six snapshots, and the average S of past values from the same weekday and 15-minute slot:

$$F(t+h) = (1 - w)\,L + w\,S, \quad w = (1 - \mathrm{e}^{-h/60}) \cdot \min(1, n/4),$$

where F is the forecast and n is the number of matching past samples. The further ahead we look, and the more history there is, the more weight the seasonal pattern gets. The LSTM [17] is optional. Its input is the last 16 resampled steps, which covers four hours, with five features per step: occupancy as a fraction of capacity, and the sine and cosine of both the time of day and the day of the week. One LSTM layer with 32 hidden units is followed by a linear layer that outputs the expected fraction of capacity four steps (one hour) later. We trained it in PyTorch [19] with the Adam optimiser [25], a learning rate of 0.01 and mean squared error as the loss. Since the output is a fraction rather than a number of cars, a model trained on a lot of one size can be reused on a lot of a different size.

A car park that has just opened has no history to learn from, so we wrote a generator for synthetic occupancy data. On a weekday, occupancy climbs steeply around 08:15, dips a little at lunchtime and falls away around 17:45, following logistic curves. Saturday has a wide peak in the afternoon, and Sunday stays quiet. Each simulated day gets its own level of busyness and its own arrival and departure times, and roughly one weekday in 25 is treated as a holiday. We also add first-order autoregressive noise to mimic the way real cars tend to arrive in bursts. Demo mode loads two weeks of this data, and the LSTM can be pre-trained on it until real data starts coming in.

3.6. Storage, Alerts and Security. All data is stored in SQLite. Since SQLite ships with Python, there is no separate database server to set up. The registered vehicles, the cars inside and their space numbers, every gate event, the occupancy history and the dashboard settings all live in a single file, so nothing is lost when the system restarts. Alerts for stolen or blocked vehicles, or for a full lot, can be sent to a webhook and by e-mail. Each one is formatted for the place it will be read. ntfy gets a phone push notification with an urgency level, Slack and Discord get a formatted message, and the e-mail is an HTML message listing the plate, the gate, the time and what happened. If a dashboard password is set, every page, API call, video stream and live update sits behind a sign-in that uses an HMAC-signed session cookie. Scripts can send a bearer token instead, and repeated failed sign-ins are slowed down.

3.7. Operator Dashboard. The dashboard is one web page, served by the backend and kept up to date over a WebSocket. It is split into four pages, and we tried to give each page a single job. *Overview* shows the number of free spaces, a map of the lot, the key figures, the last 24 hours and the latest events. A column on its right holds both gate cameras with their controls, the barrier buttons and the form for manual entries. *Activity* has the searchable history, CSV export and a seven-day report, *Vehicles* is the registry, and *Settings* holds the rest. There is also a presentation mode (Figure 7) that puts the overview on the whole screen in larger text, for a projector or a display at the entrance. The live camera views stay, but every control is hidden. A separate entry screen tells each arriving driver which space to park in. Colour is only used when it means something (green for free or allowed, amber for filling up, red for full, stolen or blocked), and all labels are written in plain words.

In our view the system has five main advantages:

(i) It is cheap. Each gate needs one camera board and one relay, however many spaces the lot has.

(ii) Because it knows which vehicle is which, it can control access, send alerts and keep a full history, rather than just counting cars.

(iii) It runs on a single computer with no internet connection, so a network outage does not stop it.

(iv) It is simple to operate, with a plain-language dashboard, a manual override and a presentation view.

(v) It can be extended. The stored events are enough to add billing, reservations or notifications to drivers later on.

It also has clear limitations. The count is only correct if every car is read at both gates, and one missed read leaves the occupancy wrong until an operator fixes it. The system also cannot tell when a car is parked outside the marked spaces. If a camera fails, automatic counting stops at that gate. The dashboard does show the state of each camera, though, and the lot can keep running on manual entries until the camera is repaired.

Input: video from the entry and exit cameras; vehicle registry; capacity *C*.
Output: parking events, occupancy, space numbers, barrier commands and alerts.
1. **For each gate, repeatedly do:**
2. Skip to the frame that matches the elapsed real time.
3. Detect vehicles; detect plates; read each plate crop with the OCR model.
4. Normalise each read; drop it if its confidence is below 0.4.
5. Snap it to a known plate if similarity ≥ 0.82, else require a valid plate format.
6. **If** the plate was not read twice within 10 s **then** continue.
7. **If** the same (or one-character-different) plate had an event at this gate within the cooldown **then** continue.
8. **If** gate = entry **then**
9. **If** the plate is stolen, blocked or unregistered **then** turn away, keep the barrier closed, send an alert.
10. **Else if** occupancy = *C* **then** turn away (lot full).
11. **Else** record the entry, assign the lowest free space, open the barrier.
12. **Else** (exit gate): free the car's space and open the barrier; flag the exit if no entry is on record.
13. Save the event, bump the state version so dashboards update, and record an occupancy snapshot.
14. **End**

ALGORITHM 1: Gate processing and access control.

TABLE 1: Experimental setup.

Component	Details
Computer	Laptop, 12th Gen Intel Core i7-1255U (10 cores, 12 threads), 15.7 GB RAM, no GPU used
Operating system	Windows 11 Home, Python 3.11.9
Detection	Ultralytics 8.4.138: YOLO11n (vehicles), YOLO plate detector; PyTorch 2.14 (CPU)
OCR	fast-plate-ocr 1.1.0 (cct-s-v2-global-model, ONNX Runtime 1.31); EasyOCR 1.7.2 for comparison
Backend	FastAPI 0.143, SQLite 3.45, OpenCV 5.0
Gate hardware	AI-Thinker ESP32-CAM, 5 V opto-isolated relay module (GPIO 13)
Test footage	Two supplied gate clips (1920×1080), one short car clip (420×380), Intel Open Edge Platform parking video [22] (1920×1080), two Indian traffic clips from Pexels [23, 24] (3840×2160)

4. Results and Discussion

Table 1 lists the setup we used for testing, and all processing ran on the laptop's CPU. The gate hardware had not yet been installed at a real car park, so we fed the cameras with recorded traffic videos instead; the backend treats these exactly like a live stream. We first walk through four scenarios that show how the system behaves, and then report measurements of plate-reading accuracy, processing time and forecasting error.

4.1. Scenario 1: Lot Full. We set the lot to eight spaces, filled all eight, and then had a registered car arrive at the entry. Figure 3 shows what happened. The sign reads 0 free spaces in red with the status *Lot full*, every space on the map is taken, and the newest event, in amber, reads *Turned away, the lot was full*. The barrier did not open. Running this scenario also exposed a bug: when the capacity was reduced, a car parked in a space beyond the new limit simply vanished from the map. We changed the system so that such cars are moved into free spaces and the move is saved, which means a later exit event still names the right space. An automated test now checks this case.

4.2. Scenario 2: Normal Operation. Figure 4 shows the overview for a twelve-space lot in normal use. Registered cars that arrive are let in and keep the space number they were given, and cars that leave free up their space. The barrier opens in both cases. The row of figures along the top gives the number of cars inside, how many came in today, how many vehicles were turned away, and the forecast for the next hour with its likely range. Below it, the chart covers the last 24 hours and marks the forecast and its range one hour ahead. On the right, each gate has its camera view, its controls and the time the barrier last opened.

4.3. Scenario 3: Stolen and Blocked Vehicles. Next, a car reported as stolen (DL8CAF5031) and one blocked from the lot (MH14XY9087) pulled up at the entry. The system refused both, kept the barrier closed and raised an alert for each. These events appear in red on the Activity page (Figure 5) together with the reason (*Reported stolen*, *Blocked from parking here*). The *Problems* filter shows only events like these, and they can be exported as CSV.

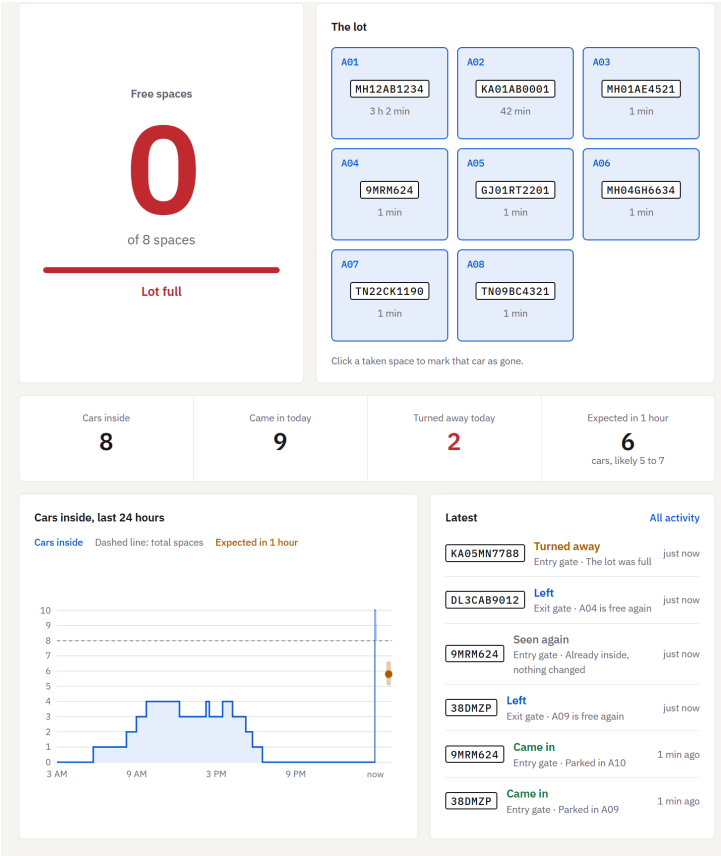


FIGURE 3: Scenario 1: the lot is full and an arriving car is turned away.

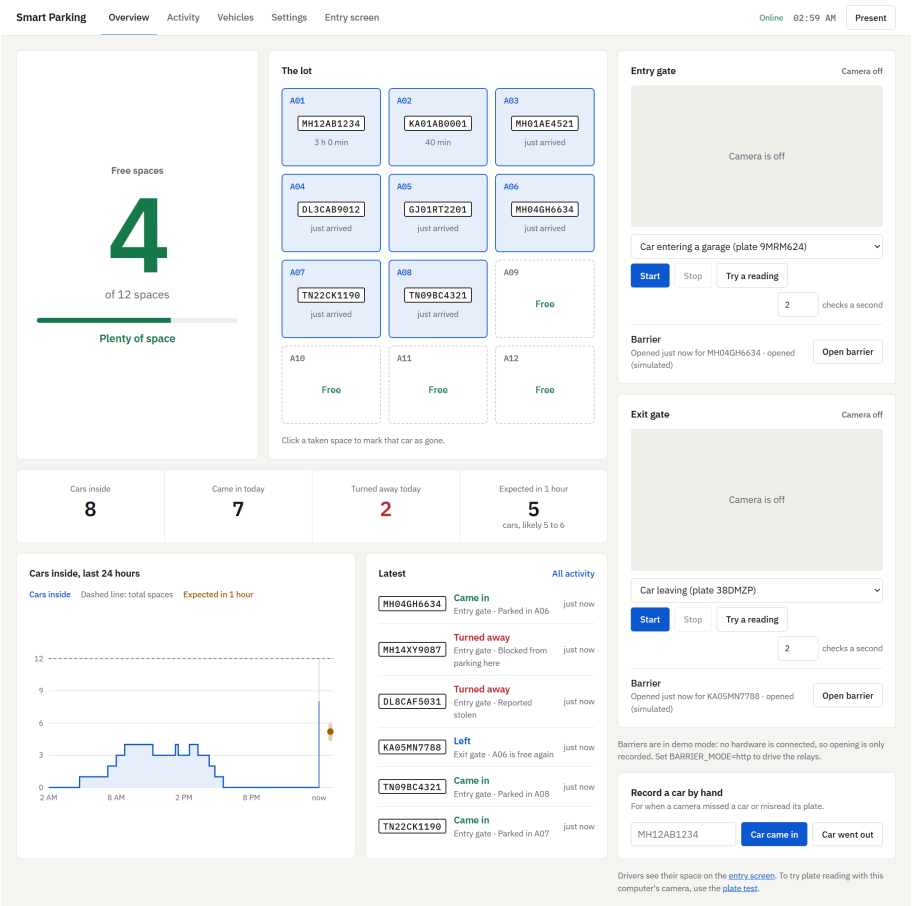


FIGURE 4: Scenario 2: overview page during normal operation of a twelve-space lot, with the two gates in the right-hand column.

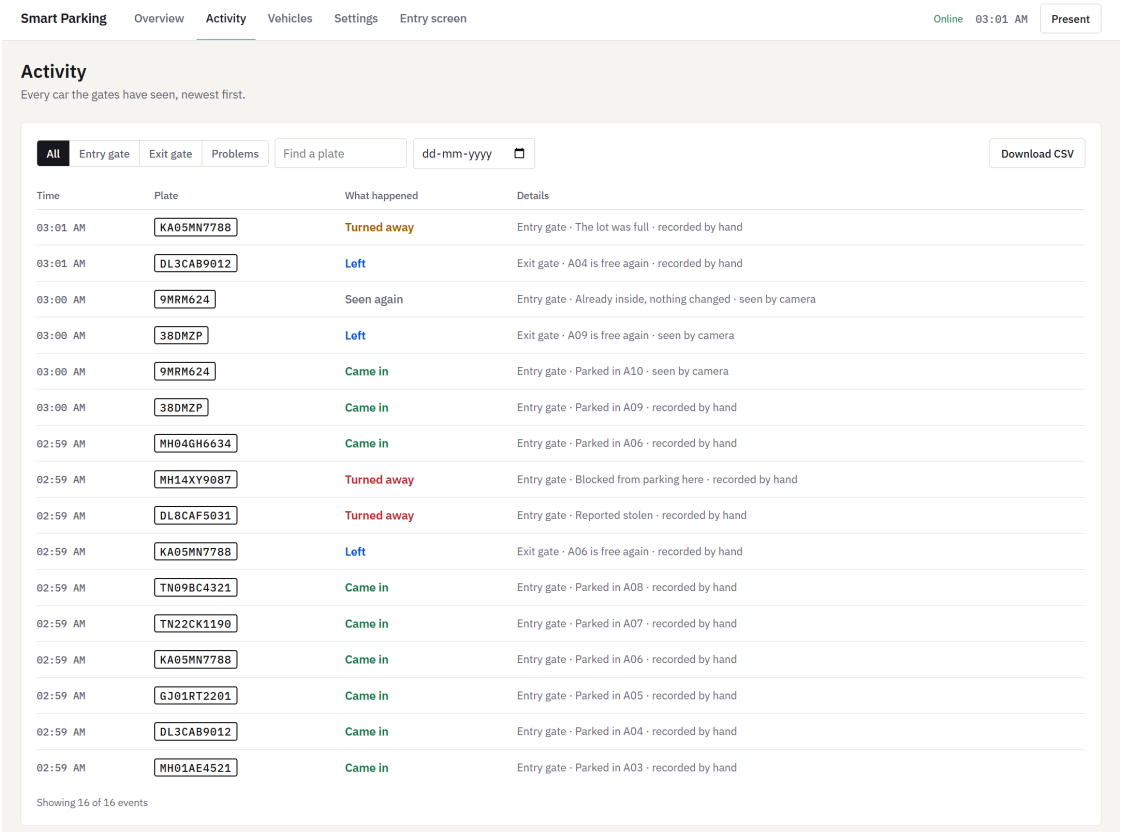


FIGURE 5: Scenario 3 and 4: Activity page with turned-away stolen and blocked vehicles, a lot-full rejection, and the camera events from Scenario 4.

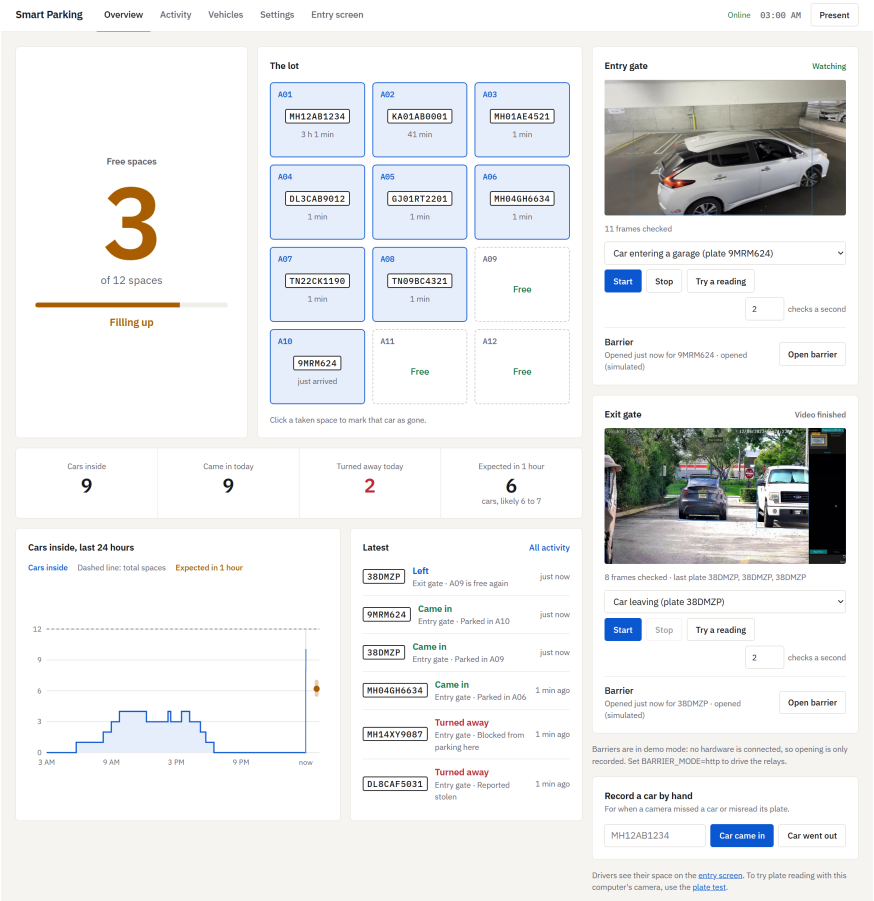


FIGURE 6: Scenario 4: both gates analysing real footage at the same time, shown in the right-hand column of the overview. Vehicle boxes are blue and plate boxes yellow; the barrier line shows the last opening.

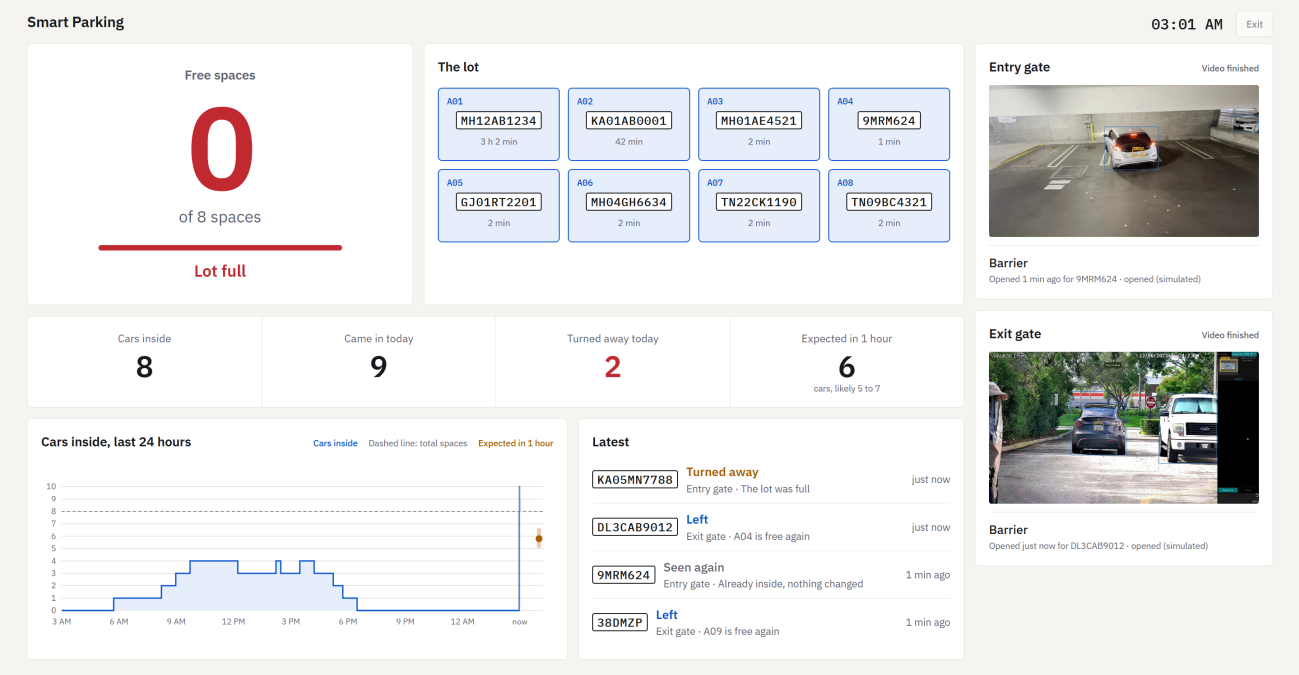


FIGURE 7: Presentation mode for a projector or a screen at the entrance: the overview with larger text and live gate cameras, without any controls.

4.4. Scenario 4: Real Footage at Both Gates. For this scenario we switched the detector from demonstration mode to YOLO11. Loading and warming up the models took between 6 and 12 s (10.5 s in the run shown here). We then gave the entry camera the Intel parking video [22], which is 30.8 s long and shows a car with plate 9MRM624 driving into a garage. The exit camera got an 11.3 s clip of a car with plate 38DMZP. Both were started at the same time and checked twice per second. Beforehand, both plates had been registered and 38DMZP had been marked as already inside. Figure 6 shows the overview during the run, with both camera views in the right-hand column.

Both videos had finished within 31.5 s of starting, so the two gates kept pace with real time even though they shared one CPU. The entry worker analysed 36 frames and skipped 718, and the exit worker analysed 8 and skipped 332. Only two changes of state were recorded. 9MRM624 came in and was given a space, and 38DMZP left and freed its space; each time, the right barrier opened. The entering car stayed in view for longer than the 15 s cooldown, so it was picked up again and logged as *already inside*, which has no effect on the count. There was no false entry or exit. We repeated the run twice more and got exactly the same events.

4.5. Plate Recognition Accuracy. To measure accuracy, we took a frame every 0.5 s from six videos (at most 60 frames per video, 189 frames in all) and ran them through the vehicle and plate detectors. Together these returned 95 plate boxes, and we went through every crop by hand. Of the 95, 75 were plates a person could read, covering 8 different plates. Another 15 were plates too blurry, tilted or cut off to read, and the last 5 were not plates at all (a motorcycle badge and a sticker were among them). We then gave each readable crop to both fast-plate-ocr and EasyOCR. The results are in Table 2, and Figure 9 shows some examples.

fast-plate-ocr read 78.7% of the readable crops exactly right and 86.7% with at most one wrong character, for a character error rate of 6.5%. EasyOCR managed only 9.3%. It did well on MP04RB0693, a large, clean Indian plate (7 of 8), but failed on the rest, usually because it mixed decorative text around the plate into its answer; for example, it read 38DMZP as “J81GMZP”. The plate model was also between 6 and 30 times faster (Table 3). Most of its mistakes came from three kinds of plate: the TN plates, taken from heavily compressed footage with a colour tint; a two-line plate with thin characters (MP04ZU2506); and a plate with a red border (MP34CA5252). The model we used is a general one, not trained specifically on Indian plates, so fine-tuning it on Indian data is the obvious next step.

For a parking system, though, the question that matters is whether a misread ever turns into a false event. Of the 16 wrong reads that passed the confidence threshold, only 3 fitted the Indian plate format, and none of those 3 came up a second time within the consensus window. In other words, **no false entry or exit would have been recorded**. Four of the eight plates were confirmed correctly by two matching reads. For the other four the system would simply have recorded nothing, and the operator would have had to add the car by hand. Had we used the looser foreign-plate format, two wrong plates would have been confirmed, which shows that the format check earns its place.



FIGURE 8: Pipeline output on real footage: vehicle boxes (blue) and plate boxes (yellow) with the plate text and confidence. Right: the badge of an auto-rickshaw is detected as a plate, but produces no valid read.



FIGURE 9: Examples of plate crops with the hand label and the fast-plate-ocr read. Top row: correct reads. Bottom row: failure cases and a non-plate detection.

TABLE 2: Plate recognition accuracy on hand-labelled crops.

Measure	fast-plate-ocr	EasyOCR
Readable plate crops (8 distinct plates)	75	75
Exact reads	59 (78.7%)	7 (9.3%)
Reads within one character	86.7%	13.3%
Mean character error rate	6.5%	67.7%
Exact reads by plate: 38DMZP / 9MRM624	22 of 23 / 30 of 31	0 of 23 / 0 of 31
Exact reads by plate: MP04RB0693 / FL020EH	5 of 8 / 2 of 3	7 of 8 / 0 of 3
Exact reads by plate: TN plates / MP04ZU2506 / MP34CA5252	0 of 6 / 0 of 2 / 0 of 2	0 of 6 / 0 of 2 / 0 of 2
Wrong reads passing format check (Indian mode)	3 of 16	–
False events after the two-read consensus	0	–

TABLE 3: Mean processing time per stage in milliseconds on the laptop CPU (after warm-up).

Video (resolution)	Frames	Vehicle detection	Plate detection	fast-plate-ocr (per plate)	EasyOCR (per plate)
Entry clip (1920×1080)	55	53.6	47.6	29.1	368.4
Exit clip (1920×1080)	23	63.9	48.4	25.1	395.4
Car clip (420×380)	8	166.0	115.6	20.0	147.2
Intel parking (1920×1080)	60	85.7	60.2	23.5	323.9
India highway (3840×2160)	29	81.7	64.6	32.8	1019.2
India street (3840×2160)	14	71.9	64.9	23.2	611.4

4.6. Processing Speed. Table 3 gives the average time taken by each stage. On full-HD and 4K frames, vehicle detection needed 54 to 86 ms per frame and plate detection 48 to 65 ms. YOLO scales every frame to the same input size, so a 4K frame costs barely more than a full-HD one. The short low-resolution clip had only eight frames, each with many vehicles, so its averages are higher and less reliable. Reading a plate took 20 to 33 ms with fast-plate-ocr, compared with anywhere from 147 to 1019 ms with EasyOCR. Altogether, a frame with one plate takes roughly 0.13 to 0.18 s. One CPU can therefore handle three to four frames per second at each of two gates. That is more than the two checks per second needed for cars creeping through a barrier.

4.7. Occupancy Forecasting. We generated 56 days of synthetic occupancy for a lot with 50 spaces; Figure 10 shows two weeks of it. The models were trained on the first 49 days and tested on the last 7. For the LSTM this meant 4,685 training windows and 300 epochs, which took about 25 s on the CPU. At each 15-minute step of the test week, the models predicted from the six hours that came before. Table 4 lists the mean absolute errors. One hour ahead, the LSTM was off by 1.79 spaces on average, compared with 2.45 for the seasonal baseline and 4.24 for a persistence forecast that assumes nothing changes. As Figure 11 shows, both models follow the daily pattern, but the LSTM picks up the morning rise and the evening fall sooner. At 15 minutes persistence actually does best, since occupancy hardly moves in that time; the LSTM was only trained for the one-hour horizon that the dashboard uses. We should stress that these results come from synthetic data. They show that the models work, not how accurate they would be at any particular car park, and the models will need retraining once real history is available.

4.8. Software Verification. The backend has 55 automated tests, which take around 17 seconds to run. They cover the summary and events API, stable space assignment, the capacity and lot-full rules, stolen and blocked vehicles, and the consensus and cooldown rules, including one-character misreads. They also check that data and settings survive a restart, that sign-in works, that the barrier relay behaves correctly against a local HTTP server, and that webhook alerts are sent. Finally, they run the camera worker on a generated video (including the case where analysis is slow and frames have to be skipped) and test the forecasting models and the synthetic data generator.

4.9. Comparison. Table 5 sets our system alongside related work, comparing method, equipment, accuracy and cost. The figures for the other systems are the ones their authors reported, measured on different data, so they cannot be compared directly. Some differences are clear all the same. Unlike the sensor-based systems [2, 4, 5], ours needs no hardware in each space, and unlike the overhead-camera systems [3, 6], it does not need to see every space. It also knows which vehicle is which, and that is what makes access control and alerts possible. The hardware comes down to two camera boards, two relays and one computer, with no internet connection required.

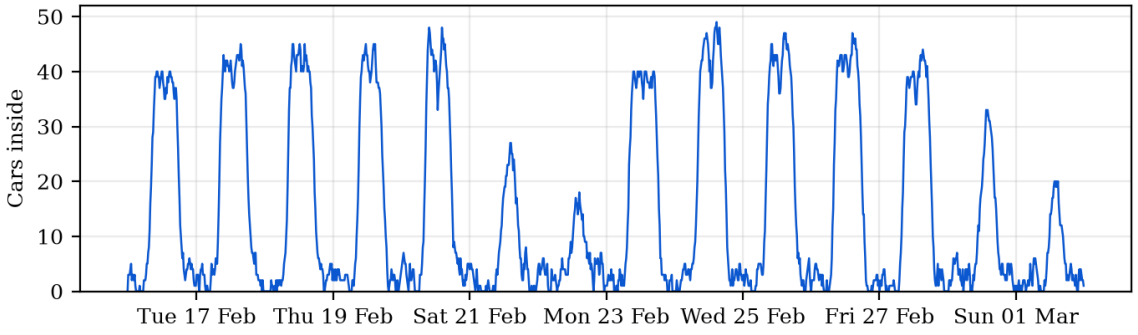


FIGURE 10: Two weeks of synthetic occupancy for a 50-space lot used for training.

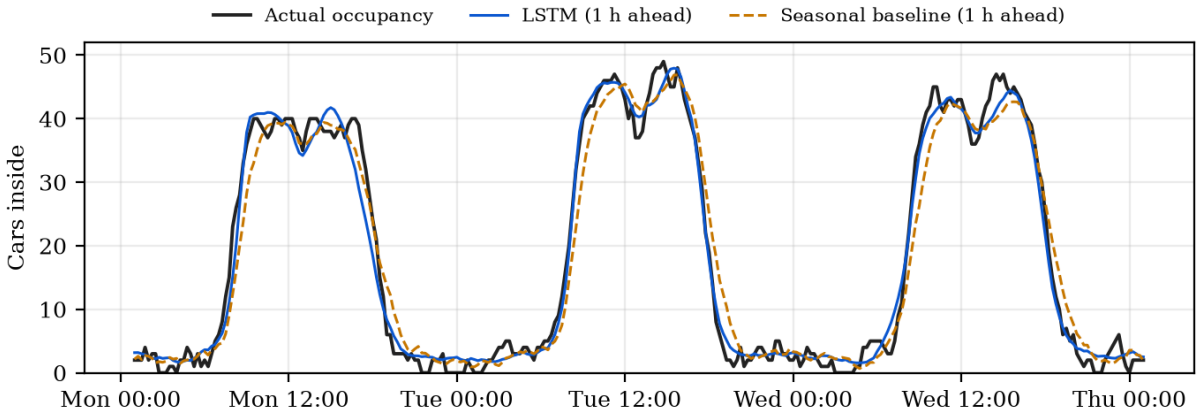


FIGURE 11: Actual occupancy and one-hour-ahead forecasts on three held-out days.

TABLE 4: Mean absolute forecasting error in spaces on the held-out week (50-space lot). The LSTM is trained for the 1-hour horizon only.

Forecast horizon	LSTM	Seasonal baseline	Persistence (no change)
15 minutes	–	1.90	1.46
30 minutes	–	2.26	2.45
1 hour	1.79	2.45	4.24

TABLE 5: Comparative evaluation with related work (figures as reported by each work).

Work	Methodology	Utilized equipment	Accuracy	Cost
Patil et al. [2]	IR sensing at entry and exit	Arduino, IR sensors, Wi-Fi module	Not mentioned	Expensive
Bassam and Samann [3]	Optical character recognition of labelled spaces	Camera, web and e-mail server, database, internet	>90%	Expensive
Nadipalli et al. [4]	IR sensing with cloud server	IR sensors, DC motor, Wi-Fi modem, microcontroller	Not mentioned	Expensive
Subramanian and Subashini [5]	IoT sensors and mobile application	Database, internet, phone, wireless sensors, server	Not mentioned	Expensive
Alsheikhy et al. [6]	CNN (AlexNet) on an overhead image	Single overhead camera	>97% (simulation)	Cheap
Proposed system	YOLO11 + plate detector + plate OCR at the gates; consensus rules; LSTM forecast	One ESP32-CAM and relay per gate, one computer	78.7% exact plate reads per frame; no false events (Section 4.5)	Cheap

5. Conclusion

In this report we described a smart parking system that recognises vehicles at the entry and exit gates using one inexpensive camera per gate. The recognition pipeline combines YOLO11 for vehicles, a dedicated plate detector and an OCR model built for plates, backed by the two-read rule, the format check and fuzzy matching. It read 78.7% of readable plate crops exactly at around 25 ms per plate, and not one misread became a false event. With real-time scheduling, two gates could process real footage at the same time on a laptop CPU. The system coped with a full lot, stolen and blocked vehicles, fixed space numbers, barrier control and alerts. On synthetic data, the LSTM gave better one-hour forecasts than the seasonal baseline, and the dashboard and its presentation view make the state of the lot easy to read at a glance.

There is still work to do. The next step is to flash the ESP32-CAM boards, install them at a real car park and measure accuracy with those cameras. We also plan to fine-tune the plate OCR on Indian plates, particularly two-line and tinted ones, and to retrain the forecaster on real occupancy data. Finally, the stored events make it possible to add features such as billing, reservations and notifications for drivers.

Data Availability

The Intel parking video is available from the Open Edge Platform repository [22], and the Indian traffic videos from Pexels [23, 24] under the Pexels licence. The hand labels, benchmark results and forecasting results are included with the project in `docs/paper_figures`.

Conflicts of Interest

The author declares that there is no conflict of interest regarding this work.

Acknowledgments

[Add acknowledgments here.]

References

[1] C. Biyik, Z. Allam, G. Pieri et al., “Smart parking systems: reviewing the literature, architecture and ways forward,” *Smart Cities*, vol. 4, no. 2, pp. 623–642, 2021.

[2] V. Patil, A. Randive, and D. Gaikwad, “Smart car parking system,” *Open Access International Journal of Science and Engineering*, vol. 6, pp. 45–47, 2021.

[3] R. Bassam and F. Samann, “Smart parking system based on improved OCR model,” *IOP Conference Series: Materials Science and Engineering*, vol. 978, no. 1, article 012007, 2020.

[4] L. S. P. S. Nadipalli, K. P. Kishore, B. M. Reddy, K. V. Kishna, and D. R. S. Aravind, “A cloud based smart parking system using IoT technology,” *International Journal of Recent Technology and Engineering*, vol. 8, no. 5, pp. 96–100, 2020.

[5] K. Subramanian and T. S. Subashini, “Ticket based smart parking system using mobile application in vehicular network,” *European Journal of Molecular and Clinical Medicine*, vol. 7, no. 3, pp. 685–695, 2020.

[6] A. A. Alsheikhy, T. Shawly, Y. F. Said, and H. Lahza, “An intelligent smart parking system using convolutional neural network,” *Journal of Sensors*, vol. 2022, Article ID 7571716, 2022.

[7] A. Alharbi, G. Halikias, M. Yamin, and A. A. Abi Sen, “Web-based framework for smart parking system,” *International Journal of Information Technology*, vol. 13, no. 4, pp. 1495–1502, 2021.

[8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, vol. 25,

2012.

[9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 779–788, 2016.

[10] G. Jocher and J. Qiu, “Ultralytics YOLO11,” 2024, https://github.com/ultralytics/ultralytics.

[11] Y. Zhao, W. Lv, S. Xu et al., “DETRs beat YOLOs on real-time object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[12] R. Laroca, E. Severo, L. A. Zanlorensi et al., “A robust real-time automatic license plate recognition based on the YOLO detector,” in *Proceedings of the International Joint Conference on Neural Networks*, 2018.

[13] S. M. Silva and C. R. Jung, “License plate detection and recognition in unconstrained scenarios,” in *Proceedings of the European Conference on Computer Vision*, pp. 580–596, 2018.

[14] “fast-plate-ocr: lightweight license plate OCR,” GitHub repository, https://github.com/ankandrew/fast-plate-ocr.

[15] JaidedAI, “EasyOCR,” GitHub repository, https://github.com/JaidedAI/EasyOCR.

[16] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, “Character region awareness for text detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9365–9374, 2019.

[17] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[18] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., OTexts, Melbourne, 2021.

[19] A. Paszke, S. Gross, F. Massa et al., “PyTorch: an imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[20] S. Ramírez, “FastAPI,” https://fastapi.tiangolo.com.

[21] Espressif Systems, “esp32-camera,” GitHub repository, https://github.com/espressif/esp32-camera.

[22] Intel Open Edge Platform, “edge-ai-resources: ParkingVideo.mp4,” https://github.com/open-edge-platform/edge-ai-resources.

[23] Pexels, “Motorcyclists commuting on Indian highway,” video 34395449, https://www.pexels.com/video/motorcyclists-commuting-on-indian-highway-34395449/.

[24] Pexels, “Motorcyclist riding through urban Indian street,” video 34394431, https://www.pexels.com/video/motorcyclist-riding-through-urban-indian-street-34394431/.

[25] D. P. Kingma and J. Ba, “Adam: a method for stochastic optimization,” in *Proceedings of the International Conference on Learning Representations*, 2015.